Cálculo de Área con Diseño Modular: Enfoque Descendente y Reutilización de Funciones en C++

Aramis Demaria

E.E.S.T. N.º 1 “Eduardo Ader” – Vicente López

Laboratorio de Programación (LPR) – 5.º Año 3.º A-B — Grupo A

Prof. Mansilla Muñoz York Elías

2026 — Grupo A

Resumen

Se implementa una calculadora de área del círculo bajo metodología descendente. El problema se descompone en el módulo *calcularAreaCirculo*, declarado mediante prototipo antes de *main* y definido después. La constante PI se protege con el calificador *const* para garantizar inmutabilidad en la memoria. Un filtro de consistencia rechaza radios no positivos. El programa muestra el área y representa una diana ASCII escalada. Las pruebas con radios de 3, 7.5 y 15 cm validan el cálculo y la clasificación MINI, ESTÁNDAR y GIGANTE. El diseño modular facilita la depuración y la reutilización, competencias clave para el Proyecto Integrador Anual. La metodología descendente, la modularización y la protección de constantes se analizan en el marco teórico, y las respuestas al cuestionario técnico de la guía se desarrollan de forma explícita.

Palabras clave: diseño descendente, modularización, prototipo de función, const, área del círculo, diana ASCII

Abstract

A circle area calculator is implemented under a top-down methodology. The problem is decomposed into the *calcularAreaCirculo* module, declared via a prototype before *main* and defined after it. The PI constant is protected with *const* to ensure memory immutability. A consistency filter rejects non-positive radii. The program displays the area and renders a scaled ASCII target. Tests with radii of 3, 7.5, and 15 cm validate the calculation and the MINI, STANDARD, and GIANT classification. The report includes explicit answers to the three technical control questions from the assignment guide.

Keywords: top-down design, modularization, function prototype, const, circle area, ASCII target

Introducción

El diseño descendente consiste en abordar un problema complejo dividiéndolo en módulos especializados. En C++ esto se concreta con funciones diseñadas por el programador: cada función resuelve una subtarea y la función principal coordina su ejecución. El caso de estudio es el cálculo del área definida como $A = \pi r^2$. Aunque la fórmula es trivial, el énfasis radica en la arquitectura: prototipo, invocación y definición separados, y protección de invariantes (PI). El objetivo general es aplicar el enfoque descendente para obtener un programa mantenible y verificable; los objetivos específicos son implementar el filtro de consistencia para el radio, proteger la constante PI y responder las tres preguntas técnicas de la guía de la Actividad 3.

Revisión de la Literatura

El diseño descendente va de lo general a lo particular, subdividiendo el sistema en módulos independientes. Modularizar consiste en dividir el programa en funciones para mejorar legibilidad, mantenimiento y localización de errores. El prototipo es la declaración anticipada que informa al compilador sobre nombre, valor de retorno y parámetros. El calificador *const* bloquea la variable en la memoria e impide su modificación; todo intento de reasignación produce error de compilación (Microsoft, 2024; Stroustrup, 2013). En C++, el compilador realiza una sola pasada de arriba hacia abajo, por lo que toda llamada debe estar precedida por su declaración.

El filtro de consistencia es una condición que rechaza datos fuera del dominio válido antes del procesamiento. En geometría, el radio debe ser estrictamente positivo; un radio nulo genera área nula (círculo degenerado) y uno negativo carece de interpretación euclídea. En sistemas críticos, los invariantes físicos y matemáticos se declaran como inmutables para que el compilador actúe como mecanismo de protección.

Fase	Qué hace	Ubicación en el código
Prototipo	Anuncia la función	Antes de main: double calcularAreaCirculo(double);
Invocación	Utiliza el módulo	En main: double areaFinal = calcularAreaCirculo(radio);

Fase	Qué hace	Ubicación en el código
Definición	Implementa la fórmula	Después de main: { const PI; return PI*r*r; }

Nota. Ciclo de vida de una función modular en C++.

Método

Se organizó la carpeta */calculoarea* con *src/main.cpp*, *docs/* y *capturas/*. La función principal solicita *radioEstudiante*, aplica el filtro *if (radio <= 0) return 1* para terminación controlada con código de salida 1, invoca el módulo y muestra *areaFinal*. La diana ASCII se implementa con estructuras *if / else if* según el rango del radio (≤ 5, ≤ 12 o > 12). La función *calcularAreaCirculo* declara *const double PI = 3.1415926535* y retorna *PI * r * r*. La compilación se ejecutó con *g++ src/main.cpp -o src/area.exe* y la ejecución con *.\src\area.exe*. El diagrama de flujo se conserva en *capturas/diagrama_flujo.png* y la evidencia de terminal en *capturas/compilacion_area.png*.

Resultados

Con radio de 7.5 cm el área resultó 176.715 cm², coincidente con el cálculo manual. Las clasificaciones fueron: 3 cm, MINI; 7.5 cm, ESTÁNDAR; y 15 cm, GIGANTE. El filtro rechazó los valores 0 y −5 con mensaje de error y código de salida 1. La invocación *double areaFinal = calcularAreaCirculo(radioEstudiante);* y la salida *cout << "[RESULTADO] El area ..." << areaFinal* funcionaron conforme a la especificación, y la definición con constante protegida retornó el producto correcto.

Nota. Figura 1. Terminal compilando y ejecutando con radio 7.5 (capturas/compilacion_area.png). Figura 2. Diagrama de flujo modular (capturas/diagrama_flujo.png).

Cuestionario Técnico de Control

A continuación se responden las tres preguntas técnicas de la guía de la Actividad 3, en formato APA.

Pregunta 1 — El Prototipo de la Función

Enunciado: Si borráramos la línea 15 (*double calcularAreaCirculo(double radio);*), ¿por qué el compilador no sabría qué hacer cuando intente leer la función *main*?

Respuesta: Porque C++ compila en una sola pasada de arriba hacia abajo. Al encontrar la llamada *calcularAreaCirculo(radioEstudiante)* dentro de *main* sin haber visto antes su declaración, el compilador no conoce su firma —nombre, tipo de retorno ni parámetros— y genera el error de identificador no declarado (*was not declared in this scope*). El prototipo es la declaración anticipada que establece el contrato: informa al compilador que la función existe, que recibe un *double* y que retorna un *double*. Sin ese contrato, la fase de análisis semántico no puede verificar la llamada ni generar el código de invocación, por lo que la compilación se detiene. Esta es la razón técnica por la que el diseño modular en C++ exige prototipo antes de *main* y definición después (Stroustrup, 2013; cplusplus.com, 2024).

Pregunta 2 — El Filtro de Consistencia

Enunciado: ¿Qué problema físico o geométrico ocurriría si intentáramos calcular el área de un círculo con un radio de 0 cm o de −5 cm? ¿Por qué es importante controlarlo?

Respuesta: Un radio de 0 cm corresponde a un círculo degenerado: un punto sin extensión, cuya área es 0 cm² y cuya diana carece de superficie. Un radio de −5 cm carece de interpretación en geometría euclídea: la distancia es una magnitud no negativa y un radio negativo no representa ningún objeto físico. Calcular con esos valores produciría áreas nulas o, si se eleva al cuadrado, un área positiva matemáticamente pero físicamente absurda, e invalidaría la clasificación de la diana (MINI/ESTÁNDAR/GIGANTE). El filtro *if (radioEstudiante <= 0) { cout << "[ERROR]..."; return 1; }* implementa programación defensiva: rechaza la entrada fuera del dominio válido antes del cálculo, evita la propagación de datos inválidos a módulos posteriores y comunica el error con código de salida 1 al sistema operativo, facilitando pruebas automatizadas y manejo en el Proyecto Integrador Anual.

Pregunta 3 — Proteger Constantes en Memoria RAM

Enunciado: ¿Por qué es más seguro para un sistema crítico (como el software de un avión o un sistema médico) declarar valores como inmutables (*const*) en lugar de usar variables comunes?

Respuesta: Porque *const* convierte el invariante en una garantía verificada por el compilador. El número π es un invariante universal; si se declara como *const double PI = 3.1415926535;*, el compilador bloquea el casillero de memoria y todo intento de reasignación —por ejemplo, *PI = 5.5;* por error de tipeo o por lógica defectuosa— produce error de compilación y detiene la generación del ejecutable. Con variable común, la reasignación sería silenciosa y todos los cálculos dependientes quedarían corrompidos sin advertencia, con riesgo de fallo en tiempo de ejecución. En sistemas críticos —aviónica, instrumental médico, control industrial—, un valor corrompido puede traducirse en decisiones incorrectas con consecuencias graves. La inmutabilidad, junto con el tipado estático de C++, actúa como cortafuegos de integridad en tiempo de compilación, principio recomendado por Microsoft (2024) y Stroustrup (2013) para software de alta confiabilidad.

Discusión

Los tres puntos del cuestionario convergen en el mismo principio: hacer explícitos los contratos y las invariantes para que el compilador y el programador detecten errores lo antes posible. El prototipo hace explícito el contrato de la función; el filtro hace explícito el dominio geométrico; y *const* hace explícita la inmutabilidad del invariante matemático. Sin esos tres mecanismos, los errores se desplazan a tiempo de ejecución, donde son más costosos y difíciles de depurar. La evidencia de terminal y el diagrama de flujo confirman que la arquitectura propuesta cumple esos contratos.

Conclusión

El enfoque modular se aplicó correctamente al completar la invocación, la salida y la definición con PI constante. El programa es mantenible, verificable y reutilizable, y las